# Automated API Bottleneck Detection, Diagnosis and Verified Remediation

Team guide: everything we built, step by step,
what each step gave us, and why

B.Tech CSE major project · RGUKT Basar
P. Dhanush · G. Vardhan · K. Anusha
Guide: Prof. G. Sagar

Status as of 11 November 2026: system complete, main campaign (220 runs) analysed; held-out session still to run.
Every number in this guide comes from the project's result files (data/results/) or its documentation (docs/, PROGRESS.md).

# Contents


| Part | What you will learn |
|---|---|
| A. The big picture | The problem, the idea, the research question, and the loop the whole project is built around |
| B. Building the platform (Steps 1–9) | Prototype, setup, telemetry repair, realistic load, local lab, shop API, injected faults, campaigns + pilot, frozen protocol |
| C. Building the brain (Steps 10–17) | Features L1–L4, detection, diagnosis (rules + ML), remediation, verification, closed loop, dashboard, results report |
| D. The main campaign | What the 220 runs were, what went wrong while running them and how we handled it, and the results per research question |
| E. What is left, and how to run things | Remaining work, everyday commands, where things live, glossary |


**WHY IT MATTERS**

Read Part A first, even if you skip everything else. Each step in Parts B and C has the same four boxes: **what we did**, **what we got**, **why it matters**, and an **example**. So you can stop at any step and still understand it.

# Part A · The big picture

## The problem

Web and mobile apps talk to a back end through an **API**. When an API endpoint becomes slow, users notice immediately, but finding *why* it is slow is hard. A developer usually digs through logs and dashboards by hand. The cause is often one of a few classic mistakes: a missing database index, the N+1 query pattern, sending far too much data, a cache that never hits, or heavy computation in the request path.

## Our idea: think like a doctor

| Doctor | Our system | Project stage |
|---|---|---|
| Notices the patient has a fever | Notices the endpoint is slower than its normal (healthy) self | Detect |
| Runs tests, finds the cause | Reads the telemetry (traces) and finds which bottleneck it is, with evidence | Diagnose |
| Prescribes medicine | Recommends a fix (e.g. 'add the missing index') | Remediate |
| Check-up after treatment | Runs the same load again with the fix and proves statistically it is faster | Re-run + Verify |

The loop the whole project is built around:

**Observe** → **Detect** → **Diagnose** → **Remediate** → **Re-run** → **Verify**

## The research question

**WHY IT MATTERS**

**Is request-level observability telemetry enough to identify and tell apart different types of API performance bottlenecks?** Where it is not enough, the system must say "insufficient evidence" instead of guessing.

We split it into six sub-questions (RQ1–RQ6). They come back in Part D:

| RQ | Question in plain words |
|---|---|
| RQ1 Detection | Can we tell a slow API apart from normal ups and downs? |
| RQ2 Diagnosis | Once it is slow, can we say which of the five bottlenecks caused it? |
| RQ3 Sufficiency | How much telemetry do we need for that: client timings only, or deep tracing? |
| RQ4 Rules vs ML | Do simple explainable rules do as well as Random Forest / XGBoost? |
| RQ5 Uncertainty | Can the system say "I don't know" instead of guessing? |
| RQ6 Remediation | Does the recommended fix work, and does the endpoint go back to normal? |

## Why we built our own API (instead of studying a real one)

To measure whether a diagnosis is right, we must **know the true cause**. In a real company nobody knows for sure why something got slow. So we built a realistic shop API and **injected each bottleneck on purpose**, with a label that only the evaluation can see. Then we can count exactly how often the system is right.

**EXAMPLE**

We start `users.list` with a hidden N+1 fault. The system must detect it, say **N_PLUS_ONE**, show evidence ("21 queries per request instead of 1"), recommend "use one JOIN", apply the fix and prove the endpoint is fast again. It never sees the label `nplus1:20`.

# Part B · Building the platform

Steps 1–9 build everything needed to produce **trustworthy, labelled experiment data**. Without this, no detection or diagnosis result would mean anything.

## Step 1 The first prototype

Phase 1–3 · commit `911af77`

**WHAT WE DID**

- A small Express (Node.js) API instrumented with **OpenTelemetry**, which records a *span* (timed step) for every request.
- A Python **workload runner** that sends requests to the API and records their timings.
- A Python **telemetry analyzer** that turns the raw records into features, keeping labels separate (leakage checks).

**WHAT WE GOT**

A working pipeline end to end: 11 runs, 320 requests, all analysed. The data of 9 Nov was frozen and checksummed (`data/raw/legacy-2026-11-09/`).

**WHY IT MATTERS**

It proved the pieces fit together. But reviewing it showed problems that would make results untrustworthy (Step 3), so we did not trust these numbers as evidence. Honest note from the status file: *"These numbers show that the pipeline works end to end. They are not evidence that bottleneck detection works."*

## Step 2 Development setup on Windows

Phase 0 · commit `15c1c39`

**WHAT WE DID**

- Made the project run on Windows (Git Bash + WSL2) as well as Ubuntu.
- Forced LF line endings for shell scripts (`.gitattributes`), because Windows CRLF breaks bash.
- Added **GitHub Actions CI** that runs every test suite on each push, and `scripts/test-all.sh` to run them locally.

**WHAT WE GOT**

Any teammate can clone, run `scripts/test-all.sh`, and see all suites pass.

**EXAMPLE**

One bug we hit: `node --test test/` worked on Node 20 but failed on Node 22. Fixed to `node --test`. Small things like this are why CI exists.

## Step 3 Repairing the telemetry

Phase 3.5A · commit `15c1c39`

**WHAT WE DID**

- **Span parenting fix:** Express's instrumentation reset the active span on every `next()`, so our experiment span could never be the parent of the handler spans. We moved it *before* Express, into the raw HTTP server callback.
- **W3C traceparent:** the runner now sends a fresh trace ID with every request, so each client request matches **exactly one** server trace.
- Per-run span files and **runtime metrics** (event-loop delay, GC, CPU, heap, memory, requests in flight).

**WHAT WE GOT**

300/300 requests matched to their own trace in the check run. In the main campaign later: **35,612 / 35,612 (100 %)**.

**WHY IT MATTERS**

Before this fix we could only say "this run produced these spans". Now we can say "*this exact request* made 21 SQL queries". That precision is what makes diagnosis possible.

```
GET /api/users HTTP/1.1
traceparent: 00-4bf92f3577b34da6a3ce929d0e0e4736-00f067aa0ba902b7-01
X-Experiment-Run-Id: main-20261111T100109-be3c28-r000064
        └─ the server adopts this trace ID, so client and server records join exactly
```

## Step 4 Realistic load: open-loop workload and calibration

Phase 3.5B · commit `60d185a`

**WHAT WE DID**

- Changed the runner from **closed loop** (wait for a response before sending the next) to **open loop** (send on a fixed timetable, like real users), with **Poisson** arrivals generated from a seed.
- Recorded the **send lag**: how late each request left compared to its timetable.
- A **calibration** tool to check that the client itself can keep the timetable.

**WHAT WE GOT**

The client keeps the timetable up to **100 requests/s** with send-lag p99 ≤ 3.9 ms and 0 failures. Our experiments use 5 requests/s, far below that limit.

**EXAMPLE**

Imagine a ticket counter. In a **closed loop**, the next person only arrives when the last one leaves. If the clerk gets slow, fewer people arrive, and the slowness hides itself. In an **open loop** people keep arriving on schedule, so a slow clerk shows up as a growing queue. That is how real traffic behaves.

**WHY IT MATTERS**

If the *client* is late, we would measure our own load generator, not the API. The send lag lets us detect and exclude such runs (protocol rule: send-lag p99 > 10 ms excludes a run).

## Step 5 A local experiment lab (instead of the cloud)

Phase 3.5C-1 · commit `8af1f81`

**WHAT WE DID**

- Ran PostgreSQL 16 and Redis 7 in **Docker** on this PC, with pinned image versions and fixed settings (autovacuum off, JIT off, no parallel workers) so nothing changes between runs.
- Scripts to start, stop, reset and check the lab (`scripts/lab-*.sh`), and a probe that measures round-trip times.

**WHAT WE GOT**

Database round trip p99 **0.33 ms** locally. The cloud databases (Neon, Redis Cloud) took **113–317 ms** per round trip.

**EXAMPLE**

An N+1 fault with 20 extra queries costs 20 × 0.3 ms = 6 ms locally, a clear signal. Against the cloud it would add 20 × 150 ms = 3 seconds of *network* noise, which would drown the actual effect. Decision (10 Nov): **local lab for all experiments; cloud only for normal app use.**

## Step 6 A realistic shop API with switchable variants

Phase 3.5C-2 · commit `cfa029e`

**WHAT WE DID**

- 9 operations of an e-commerce back end (users, products, orders, search, reports, dashboard).
- A **deterministic seed**: 100,000 users, 50,000 products, 1,000,000 orders, ~3 million order items (549 MB). The same data every time; reset from a template in about 3 seconds.
- A **variant registry**: each operation can run `healthy`, a fault, or its fix, chosen at start-up with `ABR_VARIANTS`. The API never reports which variant is active.
- The API itself runs **inside Docker**, next to the database.

**WHAT WE GOT**

Healthy operations answer in 4–12 ms. Running the API inside Docker cut database span time from 0.90 ms to **0.47 ms**, because Docker Desktop's port forwarding added about 1 ms per query.

**EXAMPLE**

Starting `users.list` with an injected N+1 fault (20 rows):

```
ABR_VARIANTS='{"users.list":"nplus1:20"}' scripts/lab-api.sh up
```

**WATCH OUT**

Small details mattered. Search used `lower(name)` without a matching index and took 42 ms when healthy. With a C-collation expression index it takes 0.09 ms. Reports use a **fixed reference date** instead of `now()`, so results never drift from day to day.

## Step 7 Injecting the five bottlenecks (and their fixes)

Phase 3.5C-3 · commit `7c2f018`

**WHAT WE DID**

Each bottleneck class was built into **two different operations**, at several **severities**, each with a reviewed **fixed** variant that returns exactly the same data (so before/after comparisons measure only the fix).

| Class | Variant | What the faulty code does wrong | The fix |
|---|---|---|---|
| DB_BOTTLENECK | noindex | the index the query needs is dropped, so PostgreSQL scans the whole table | recreate the index |
| N_PLUS_ONE | nplus1:N | one query for the page, then one more query per row | one query with a JOIN / subquery |
| LARGE_PAYLOAD | payload:N | ignores pagination: N rows with every column | one page, summary columns |
| CACHE_OPPORTUNITY | cache | cache key changes on every request, so it never hits | stable key, 60 s TTL |
| CPU_BOTTLENECK | cpu:N | expensive PBKDF2 'signature' (N thousand iterations) per response | ordinary hash |

**EXAMPLE**

The N+1 fault, taken from `target-api/src/routes/domain.js`:

```
-- faulty: 1 query for the page ...
SELECT id, name, city FROM users ORDER BY id LIMIT $1 OFFSET $2
-- ... then, for EACH of the 20 users, one more query:
SELECT count(*)::int AS n FROM orders WHERE user_id = $1

-- fixed (= healthy): everything in one query
SELECT u.id, u.name, u.city,
       (SELECT count(*)::int FROM orders o WHERE o.user_id = u.id) AS order_count
  FROM (SELECT id, name, city FROM users ORDER BY id LIMIT $1 OFFSET $2) u
```

**WHAT WE GOT**

**10/10** fault checks pass (each fault active when on, gone when fixed) and **3/3** severity ladders grow as expected. Example: N+1 25.2 ms vs 4.0 ms fixed, 21 vs 1 queries per request; payload 3.09 MB vs 1.7 kB.

**WATCH OUT**

Calibration changed two faults: CPU severity 20 added only +4 ms on this PC, so the default became 100 (+21 ms). The reports cache fault at 1 request/s with up to 90 days *collapsed* (16.9 s responses) instead of slowing down, so its workload became 0.5 requests/s with 7/14/30 days (healthy 16 ms vs faulty 441 ms).

# Step 8 Campaigns, ground-truth labels and the pilot

Phase 3.5D-1 · commit `0275bbb`

**WHAT WE DID**

- A **campaign orchestrator** (`experiments/campaign/`) that runs many experiments automatically. For every run: reset the database → start the API with one variant → send the load → save database statistics → stop the API → record progress.
- The run order is **shuffled** with a seed. The truth ("this run was nplus1:20") goes only to `data/labels/`, never into the features.
- A **pilot campaign** of 48 runs to measure noise and effect sizes before deciding anything.

**WHAT WE GOT**

- 48/48 runs completed; 7,459/7,459 requests matched to their traces.
- Healthy run-to-run noise (CV): **server-side 0.2–8.3 %** vs **client-side 4–24 %**.
- All 10 faults clearly separated from healthy on server-side latency (×2.0 for the weakest up to ×87.9 for the reports cache).

**WHY IT MATTERS**

The pilot told us **where to measure** (server side: 3× less noise than the client) and **how many repeats** we need. These numbers set the size of the main campaign and the detection threshold. We decided them before seeing the main data.

**EXAMPLE**

Why server-side? The client measurement includes the network and Python's own overhead (a floor of about 15 ms). Illustration: a 3 ms slowdown on a 4 ms endpoint is obvious on the server (×1.75) but nearly hidden on the client (about 19 ms vs 16 ms, within the client's noise).

# Step 9 Freezing the experiment protocol (v1)

Phase 3.5D-2 · commit `4c0f60d`

**WHAT WE DID**

Wrote `experiments/protocol.md`: every rule decided **before** the main data existed, adopted by the team on 11 Nov (to be presented to Prof. Sagar). Any later change must be logged as a dated **amendment** with a reason. The rules are also **enforced in code**.

| Rule | Why |
|---|---|
| Anomalous = server p50 > max(P99 of healthy runs, 1.2 × healthy median), per operation | absorbs normal noise; ignores slowdowns under 20 % |
| Main campaign 220 runs; held-out session 68 runs on another day | sized from the pilot noise |
| False alarms must be ≤ 5 %; 'back to healthy' means within ±10 % | success criteria fixed in advance |
| Clean git tree before a campaign; one discarded warm-up run | traceable code; the first run is the noisiest |
| Exclude a run if: incomplete tracing, > 1 % failed requests, send-lag p99 > 10 ms, different code/image, on battery (added later) | the run did not measure what it should; re-run it once |

**WHY IT MATTERS**

This is what makes the results **credible**. If we chose thresholds after looking at the results, we could make any method look good. Freezing the rules first, and testing finally on a held-out day, prevents that.

# Part C · Building the brain

Steps 10–17 turn raw telemetry into decisions: *is it slow? why? what fix? did it work?*




## Step 10 Features in four telemetry levels (Analyzer v2)

Phase Analyzer v2 · commit `committed`

**WHAT WE DID**

`pipeline/features/` summarises each run into **30 numbers**, grouped into four cumulative **telemetry levels**. The levels are the heart of RQ3: how deep must you look?

| Level | Adds | Example features |
|---|---|---|
| L1 client only | what any load-testing tool sees | client latency p50/p95/p99, response size, error rate |
| L2 + server timing | the API's own request span | server latency p50/p95/p99, client–server gap |
| L3 + runtime | process signals | CPU per request, event-loop delay, GC time, heap, requests in flight |
| L4 + DB/cache tracing | a span for every SQL query and Redis call | queries per request, repeated identical queries, DB time and share, cache hit ratio, time in app code |

**EXAMPLE**

One N+1 run at L4: `db_queries_per_request_mean = 21.0`, `repeated_query_max_median = 20`. The healthy run of the same endpoint: 1.0 and 1. At L1 the same run only shows "client latency 34 ms instead of 14 ms". It is slow, but nothing says why.

**WHAT WE GOT**

Features never contain the route, variant or any ID (enforced by leakage tests). Database time is computed as the *union* of overlapping query spans, so parallel queries are not double-counted.

## Step 11 Detection: is this run slower than normal?

Phase 4 · commit `committed`

**WHAT WE DID**

- `pipeline/detection/rule.py` implements the frozen protocol rule, per operation.
- The baseline comes from the healthy runs of the same operation. A healthy run being scored is left out of its own baseline (**leave-one-out**).
- The output is **anomalous**, **normal** or **inconclusive** (excluded run, < 95 % of requests traced, < 2 healthy runs), always with its reason.
- Only `pipeline/evaluation/labels.py` may read labels, and tests enforce that.

**EXAMPLE**

Worked example, `users.list` in the main campaign:

```
healthy median            = 4.45 ms
1.2 x healthy median      = 5.34 ms
P99 of healthy run p50s   < 5.34 ms   (healthy runs are very consistent here)
threshold = max(P99, 5.34) = 5.34 ms

run with hidden nplus1:20 : server p50 24.05 ms  > 5.34 ms  -> ANOMALOUS (x5.4)
a typical healthy run     : server p50 ~4.5 ms   < 5.34 ms  -> normal
```

**WHAT WE GOT**

Main campaign: false alarms on **4/80** healthy runs (5.0 %), faults detected in **103/110** runs (93.6 %). Details in Part D.

## Step 12 Diagnosis: which bottleneck is it?

Phase 5 · commit `6aa367a`

**WHAT WE DID**

Step 1: make every feature **relative to the healthy baseline** of the same endpoint (log ratios, or differences for shares). "21 queries" means nothing alone; "20 more queries than normal" is the signal. That also lets a model trained on some endpoints work on an endpoint it has never seen.

**Method A: mechanism rules** (`pipeline/diagnosis/rules.py`). One rule per class, written from how each fault works:

| Class | Rule fires when (compared with healthy) |
|---|---|
| N_PLUS_ONE | ≥ 2 more queries per request AND one identical query repeated ≥ 2 more times |
| DB_BOTTLENECK | DB time ≥ 2× (and ≥ 2 ms more), DB share +15 points, with the SAME number of queries |
| LARGE_PAYLOAD | response ≥ 4× larger |
| CACHE_OPPORTUNITY | cache hit ratio ≥ 0.5 lower than a cache-using baseline |
| CPU_BOTTLENECK | time in application code ≥ 2× (and ≥ 3 ms more), same queries, response < 2× larger |
| (no rule fires) | INSUFFICIENT_EVIDENCE: the system says it does not know |

**Method B: machine learning** (`pipeline/diagnosis/ml.py`). Random Forest (300 trees) and XGBoost (200 trees, depth 4), trained on the same relative features at each level L1–L4, with balanced class weights. They may **abstain** when their confidence is below 0.5 or 0.7.

**EXAMPLE**

What the rules print for the hidden `orders.list noindex` run in our closed-loop demo:

```
DB_BOTTLENECK (high)
  - database time 83.8 ms vs 2.1 ms healthy with the same number of queries (1.0 vs 1.0)
  - database share of the request 97% vs 44%
```

**WHY IT MATTERS**

Fair comparison: rules and ML use the same runs and splits. **Grouped** cross-validation keeps all repeats of a condition together, so a model never sees the exact condition it is tested on. **Leave-one-operation-out** hides a whole endpoint. The final test trains on the main campaign and tests once on the held-out day.

# Step 13 Remediation: what should the developer change?

Phase 6 · commit `c4855f5`

**WHAT WE DID**

A catalogue (`pipeline/remediation/catalogue.py`) maps each class to one reviewed fix, the variant that implements it, and the metric that must move if the fix really works.

| Diagnosis | Recommended fix | Must improve |
|---|---|---|
| DB_BOTTLENECK | add the missing index (then ANALYZE) | DB time ↓, DB share ↓ |
| N_PLUS_ONE | replace per-row queries with one JOIN / batched query | queries per request ↓, repeats ↓ |
| LARGE_PAYLOAD | paginate and select only the needed fields | response size ↓ |
| CACHE_OPPORTUNITY | stable cache key with a TTL | hit ratio ↑, queries ↓ |
| CPU_BOTTLENECK | remove the expensive computation from the request path | app time ↓, event-loop delay ↓ |
| INSUFFICIENT_EVIDENCE | collect more evidence (re-run with full tracing, L4) | – |

**WATCH OUT**

The system **never edits code**. It switches the endpoint to the pre-written, reviewed fixed variant, the way a developer would deploy a reviewed fix. Automatic code changes are out of scope.

**WHAT WE GOT**

Main campaign: the recommended fix matched the injected fault for **110/110** fault runs.

# Step 14 Verification: did the fix really work?

Phase 7 · commit `c4855f5`

**WHAT WE DID**

Run the same workload (same seed) again with the fix, then answer three questions (`pipeline/verification/stats.py`):

| Question | Test | In plain words |
|---|---|---|
| Did it improve? | one-sided Welch t-test on log latency, Holm correction over all fixes, α = 0.05 | is 'after' really faster, and not just luck? Holm stops us from fooling ourselves when testing 10 fixes at once |
| Is it back to healthy? | TOST equivalence: 90 % CI of fixed/healthy within ±10 % | proving 'the same as normal' needs its own test; 'no difference found' is not proof |
| Right mechanism? | the catalogue's metric moved in the expected direction | e.g. an N+1 fix must lower queries per request, not just latency |

**EXAMPLE**

users.list, faulty nplus1:20 vs its fix (main campaign): median **13.05 → 2.85 ms** (healthy 4.45), ratio ×0.212, p (Holm) 0.0043 → **improved**. Queries per request went down → right mechanism. Back to healthy: not proven, because the interval was too wide (see the charger story in Part D).

# Step 15 The closed loop: everything in one command

Phase 7 · commit `c4855f5`

**WHAT WE DID**

`experiments/campaign/loop.py` runs the whole cycle on a live API. The injected fault is revealed only at the end.

```
$ python -m campaign.loop --operation users.list --variant nplus1:20 \
      --baseline-campaign main-20261111T100109-be3c28
[1/5] inject:    users.list runs with a variant the system is not told about (3 runs)
[2/5] detect:    ANOMALOUS (3/3 runs); server p50 24.05 ms > threshold 5.34 ms
                 (x5.40 of the healthy median)
[3/5] diagnose:  N_PLUS_ONE (high); 21.0 queries per request vs 1.0 healthy;
                 one query repeated 20x per request vs 1x
[4/5] recommend: replace the per-row queries with one JOIN / batched query
                 -> restart users.list with variant nplus1-fixed:20
[5/5] verify:    server p50 24.05 -> 5.11 ms (x0.208, p 0.0000); healthy 4.45 ms
                 -> IMPROVEMENT VERIFIED
```

**WHAT WE GOT**

Two real loops, both fully successful: users.list N+1 (24.05 → 5.11 ms) and orders.list missing index (86.20 → 5.31 ms, ×0.060). This is the project's title, working end to end.

# Step 16 The dashboard

Phase 8 · commit `43608ed`

**WHAT WE DID**

A read-only **Next.js** app (`dashboard/`) over the result files. It has no database and copies no logic: every number comes from a file under `data/results/`, and each section names its source.

| Page | Shows |
|---|---|
| Overview / | headline cards (runs, false alarms, detection, diagnosis F1, fixes verified), per-endpoint table, detection per class, rules vs RF vs XGBoost per level, verification table, closed loops |
| Endpoint /endpoint | every run of one route: injected condition (for checking only), server p50 vs threshold bar, verdict (green = correct, red = wrong), diagnosis, evidence, recommended fix |
| Closed loop /loops/<id> | the 5 steps of one loop and before/after latency bars |

**EXAMPLE**

Start it:

```
cd dashboard
npm install        # once
npm run dev        # open http://localhost:3001
```

# Step 17 The results report

Phase 9 · commit `99466a5`

**WHAT WE DID**

`python -m evaluation.report --main <id> [--heldout <id>]` writes `docs/results.md`: one section per research question plus threats to validity, generated from the result files.

**WHY IT MATTERS**

Nobody types numbers by hand, so the report can never disagree with the data. After the held-out session, one command updates every number.

# Part D · The main campaign

## What the 220 runs were

A **run** is one 30–60 second experiment: one endpoint in one condition, against a freshly reset database and a freshly started API. Requests inside one run share the same process and moment, so they are not independent. That is why we count results in **runs**, not in requests.

| Group | Conditions | Repeats | Runs |
|---|---|---|---|
| Healthy | 8 operations | 10 each | 80 |
| Faults | 22 conditions (5 classes, several severities) | 5 each | 110 |
| Fixed variants | 10 fixes | 3 each | 30 |
| Total | | | 220 (+24 re-runs of excluded runs) |

| Level of data | Main campaign |
|---|---|
| Runs (experiments) | 220 used, 244 executed |
| Requests | 35,612, 0 failed, 100 % matched to a trace |
| Spans | many per request (an N+1 request alone has 21 query spans) |
| Runtime samples | 1 per second per run |
| Features | 244 rows × 30 features |

**EXAMPLE**

Total controlled data once the held-out session is done: pilot 48 + main 220 (+24) + held-out 68 + loops 12 ≈ **350 runs** and over **50,000 traced requests**. 220 is enough because the pilot showed server-side noise of only 0.2–8.3 %. The detection rate's 95 % confidence interval is a tight 87–97 %.

## What went wrong while running it, and how we handled it

Every problem was caught by the protocol's own checks, handled by its rules, and written down as an amendment. That is exactly what a protocol is for.

| Incident | Effect | What we did |
|---|---|---|
| Docker Desktop gave the API image a **new ID on every build**, even for identical code, and we rebuilt before every run | the 'same image' rule excluded 218 runs, all false alarms | verified identical layers and creation time; the rule now compares a **content hash**; no rebuild per run (amendment 4) |
| **PostgreSQL restarted once** during a database reset | 2 runs failed | completed with --resume; the reset now retries once |
| **24 runs had client send lag > 10 ms**, mostly while analysis ran on the same PC | 24 excluded (11 %) | re-run once at the end, as the protocol says; all passed. Lesson: run nothing heavy during a campaign |
| **Charger plugged in and out** 5 times (Windows power log) | on battery, healthy latency ×1.6–2.3 higher; 100 runs on charger, 137 on battery, 7 mixed | kept the campaign (team decision) and reported it as a threat; rule 5 now excludes battery runs and refuses to start on battery |

**WHY IT MATTERS**

Is the charger issue important? It is **measurement noise**, not part of the research question. The run order was shuffled, so battery runs are spread evenly across all conditions: noise, but no bias. Diagnosis was unaffected (110/110). It mainly widened the verification intervals. And it is a good viva story: we found an unexpected noise source, traced it, measured it, and added a rule to prevent it.

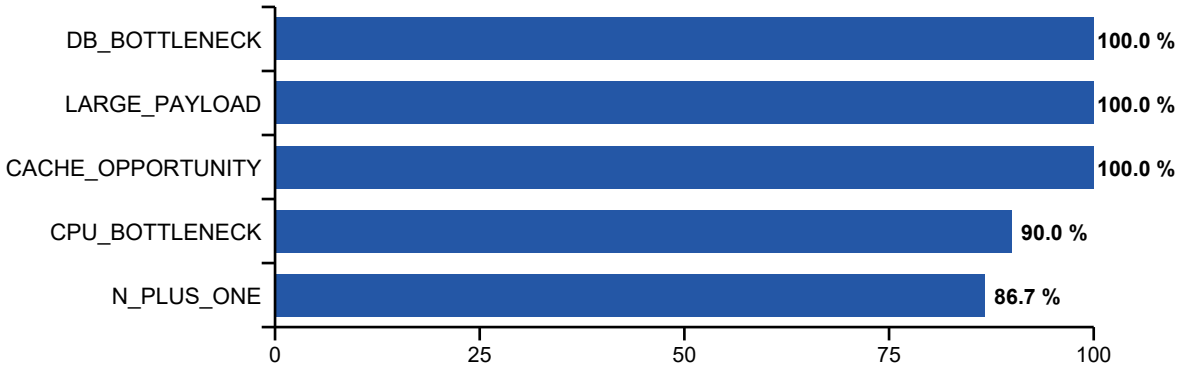
# Results per research question (development numbers)

These come from the main campaign. The **final** numbers come from the held-out session (Part E).

## RQ1 Detection: yes

False alarms on **4/80** healthy runs (**5.0 %**, 95 % CI 2.0–12.2 %), exactly at the ≤ 5 % limit. Faults detected in **103/110** runs (**93.6 %**, CI 87.4–96.9 %); precision 96.3 %. The misses are the mildest settings (users.get nplus1:3, dashboard cpu:20), whose slowdown is below the 20 % the rule is designed to ignore.

**Detection rate by bottleneck class (main campaign)**

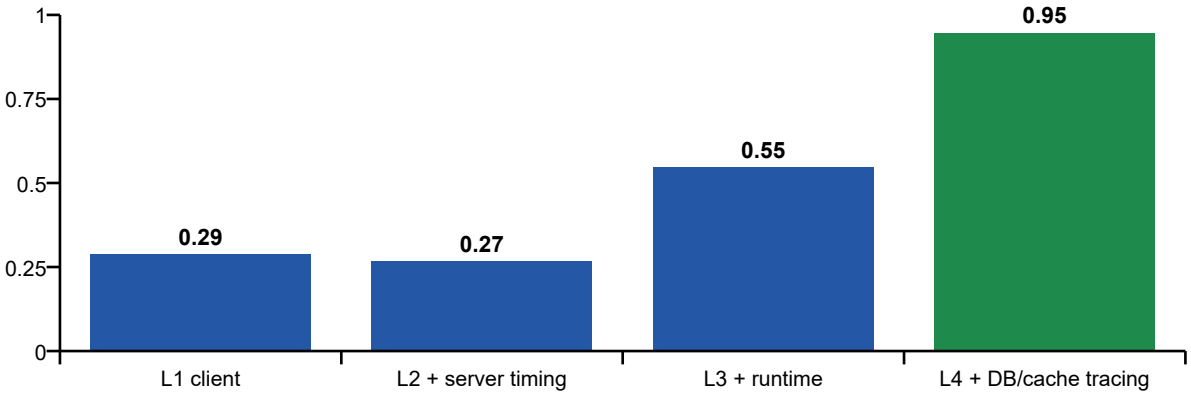



## RQ2 Diagnosis: yes, with full tracing

The mechanism rules diagnosed **110/110** fault runs correctly (macro-F1 1.000), each with written evidence. Caveat: the rule thresholds were set on the pilot, so this must be confirmed on the held-out day.

## RQ3 How much telemetry is needed? The central finding

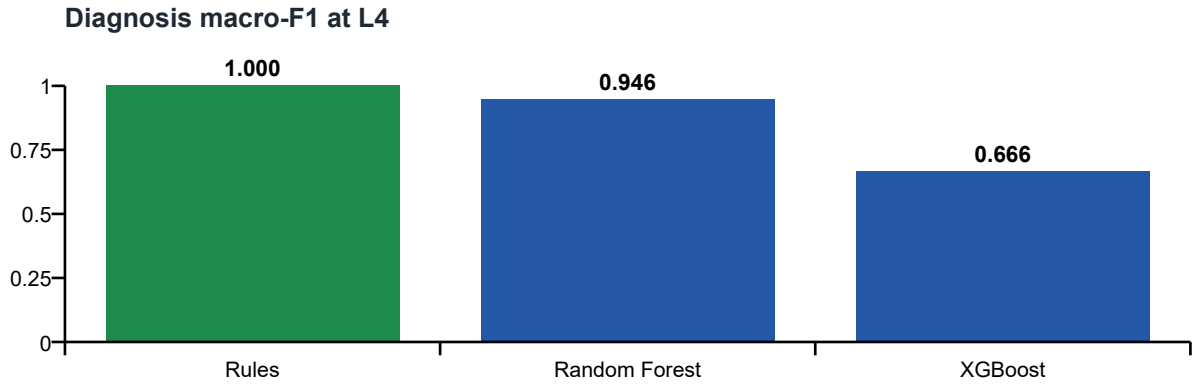
**Random Forest diagnosis quality (macro-F1) by telemetry level**




**WHY IT MATTERS**

**Latency alone tells you that an API is slow, not why.** Client or server timings give about 0.29 (not far above guessing, which is about 0.2 for 5 classes). Runtime signals help (0.55), but telling the five causes apart needs tracing of database and cache calls (0.95). With an endpoint held out completely, Random Forest at L4 still reaches 0.955.

## RQ4 Rules vs ML: the simple rules are not beaten




XGBoost struggles with CACHE_OPPORTUNITY (only 10 cache runs). The rules also explain themselves.

## RQ5 Saying "I don't know": yes

Random Forest with a confidence cut-off of 0.5 answers 84 % of runs and is right on **100 %** of those. At 0.7 it answers 57 %, also 100 % right. The rules abstain (INSUFFICIENT_EVIDENCE) when no mechanism matches.

## RQ6 Remediation: right fix always; improvement mostly shown

- Right fix recommended for **110/110** fault runs.
- **7/10** fixes proven to improve latency (Holm, α 0.05). The other 3 also got faster (×0.31–×0.50) but fell just short (p 0.051) with 5 vs 3 runs.
- **0/10** proven 'back to healthy' within ±10 %. The fixed medians are close to healthy (×0.81–1.16), but the intervals were too wide (charger noise).
- Both closed-loop demos fully verified.

# Part E · What is left, and how to run things

## Where we are

| Part | Status |
|---|---|
| System (all stages, dashboard, report) | done |
| Main campaign (220 runs) + analysis + 2 closed loops | done |
| Documentation (docs/05–09, PROGRESS.md) | done |
| Held-out session (68 runs, another day, charger plugged in) | to do (~75 min) |
| Guide's review of the protocol and results | to do |
| Thesis chapters and viva demo | to do |

## The held-out session (final numbers)

**WATCH OUT**

- **Another day** than the main campaign, on the **same laptop** (all data must come from one machine).
- **Charger plugged in** the whole time. The campaign refuses to start on battery.
- Close heavy programs. Commit everything first (git must be clean).
- Analyse it **once**. The held-out data must never be used to change a method.

```
source scripts/env.sh
# 1. the session: 68 runs, ~75 min
python -m campaign.run --config experiments/configs/campaigns/heldout.json
# 2. final detection + diagnosis (baselines and models from the main campaign)
scripts/analyze-campaign.sh <heldout-id> --train main-20261111T100109-be3c28
# 3. final docs/results.md
python -m evaluation.report --main main-20261111T100109-be3c28 --heldout <heldout-id>
```

## Everyday commands

| I want to… | Command | Docker? |
|---|---|---|
| run all tests | scripts/test-all.sh | no |
| see the dashboard | cd dashboard && npm run dev → localhost:3001 | no |
| regenerate the results report | python -m evaluation.report --main <id> | no |
| start / stop the lab | scripts/lab-up.sh / scripts/lab-down.sh | yes |
| run a closed-loop demo | python -m campaign.loop --operation users.list --variant nplus1:20 --baseline-campaign <main-id> | yes |
| continue an interrupted campaign | python -m campaign.run --resume <campaign-id> | yes |
| analyse a campaign (all stages) | scripts/analyze-campaign.sh <campaign-id> | no |

**WATCH OUT**

Docker Desktop only needs to be running for experiments (lab, campaigns, loops). The database volume survives restarts. After a reboot, open Docker Desktop and run scripts/lab-up.sh. On another laptop, git brings the code but **not** the data/ folder or the .env files: copy data/ separately.

## Where things live

| Folder | What is inside |
|---|---|
| target-api/ | the shop API, its faults and fixes, OpenTelemetry instrumentation |
| infrastructure/ | Docker Compose for PostgreSQL, Redis and the API |
| experiments/ | workload runner, campaign orchestrator, closed loop, configs, protocol.md |
| pipeline/ | features, detection, diagnosis, remediation, verification, evaluation (report) |
| dashboard/ | Next.js dashboard |
| data/ | all experiment data and results (not in git, so back it up!) |
| docs/ | guides 01–09 and the generated results.md; start at docs/README.md |
| PROGRESS.md | the single progress tracker: phases, commits, key numbers, decisions |

## Mini glossary

| Term | Meaning |
|---|---|
| p50 / p99 | median / 99th-percentile latency. 'Server p50' = the run's median server time |
| CV | coefficient of variation (spread / mean): how noisy something is |
| Span / trace | a timed record of one step (a SQL query, a Redis call) / all steps of one request |
| Baseline | the healthy runs of the same endpoint; everything is compared with it |
| Macro-F1 | accuracy-like score averaged over the 5 classes, each class counted equally (1.0 = perfect) |
| Wilson CI | the honest range for a rate such as 103/110 |
| Holm correction | keeps 'lucky' results out when testing many fixes at once |
| TOST | a test that proves two things are the same (within ±10 %), not just 'no difference found' |
| Held-out | data never used while building the methods; gives the final, honest numbers |
| Amendment | a dated, explained change to the frozen protocol |

More detail: docs/05-system-walkthrough.md (how it works), docs/06-experiments-and-data.md (the data), docs/07-results-explained.md (results + incidents), docs/08-dashboard-guide.md, docs/09-glossary-and-faq.md.